Decoding the MIPS-evolved programs: Parity-last 4, Alternating-last 4, Base-4/5/7 addition

November 2, 2026

Notation

In every table, $h = (h_0, h_1, \ldots)$ is the RNN hidden state after mapping onto its learned integer lattice; primes denote the next state; x (or the digit pair a, b) is the current input; and y is the output read from the new state. Operators: $\mathrm{lt}(u, v) = \mathbf{1}[u < v]$, $\mathrm{eq}(u, v) = \mathbf{1}[|u - v| < 0.5]$, $\mathrm{zero}(u) = \mathrm{not}(u) = \mathbf{1}[|u| < 0.5]$, $\mathrm{mod}(u, v) = u \bmod |v|$ (0 if $v = 0$), $\mathrm{xor}(u, v) = (\mathrm{round}(u) + \mathrm{round}(v)) \bmod 2$. Displayed equations are regrouped/rounded MIPS-evolved witnesses, verified exactly on the full recorded transition tables. The state meanings below were obtained by labeling each lattice state with the input histories that reach it.

1 Parity-last 4

Parity-last 4

$h_0 \in \{0,2,3,4,8\}$, $h_1 \in \{0,1,2,5,7,9\}$, $h_2 \in \{0,4\}$ (integer hidden-state coordinates; 16 joint states), $x \in \{0,1\}$ (input bit), $y \in \{0,1\}$ (output bit)

Scalar relation	Input-table size	Base	Evolved	MIPS-evolved	Discovered equation (MIPS-evolved)
$h_0' = f_0(h_0, h_1, h_2, x)$	32 state-input pairs	0/10	0/10	**1/10**	$\min(((\max(h_1, h_0) - x)h_1 - x)h_1 - x + 2,$ $4\,\lvert\mathrm{xor}(h_1(0.227249 - x), h_1) + x - \mathrm{zero}(h_1)\rvert)$
$h_1' = f_1(h_0, h_1, h_2, x)$	32 state-input pairs	**5/10**	**3/10**	**10/10**	$\lfloor\min(\min(2.98567x + 0.660363h_1,\ 2.32073)$ $+\ x\max(3.00001, h_1),\ 9.32075)\rfloor$
$h_2' = f_2(h_0, h_1, h_2, x)$	32 state-input pairs	**10/10**	**10/10**	**10/10**	$4\,\mathrm{eq}(\mathrm{lt}(h_1, 2h_0), x)$
$y = g(h_0, h_1, h_2)$	16 states	**10/10**	**10/10**	**10/10**	$\min(1, h_2)$
Subproblems recovered across seeds		**3/4**	**3/4**	**4/4**	

Task. y_t = parity of the last 4 input bits.

What the state stores. 16 states = all 4-bit windows $d\,a\,b\,c$ (oldest→newest).

- $h_2 = 4\cdot$ parity of the 4-bit window, hence $y = \min(1, h_2) = h_2/4$.
- (h_0, h_1) is a shift register of the last 3 bits $a\,b\,c$:

abc	000	100	010	110	001	101	011	111
(h_0, h_1)	$(2\|3, 0)$	$(0, 1)$	$(0, 2)$	$(4, 2)$	$(0, 5)$	$(3\|4, 5)$	$(8, 7)$	$(4, 9)$

  ($2|3$ and $3|4$: small RNN noise that depends on the bit just dropped.)
- h_1 essentially encodes the last two bits (b, c): $\{0,1\}{=}00$, $2{=}10$, $5{=}01$, $\{7,9\}{=}11$, roughly $h_1 \approx 5c + 2b + a$.

Parity rule (h_2') — the real algorithm. The new window is $a\,b\,c\,x$, so new parity = $\mathrm{parity}(a, b, c) \oplus x$, independent of the old h_2. On this embedding the 3-bit parity is *linearly separable*: $h_1 < 2h_0 \iff \mathrm{parity}(a, b, c) = 0$ (true on all 10 distinct (h_0, h_1)). Then $\mathrm{eq}(\cdot, x)$ is XNOR, giving $4\,(\mathrm{parity} \oplus x)$.

Shift of h_1. The $2.98567x$ term is dead (the inner min saturates when $x = 1$). Effectively

$$h_1' = \begin{cases} \lfloor\min(0.66h_1, 2.32)\rfloor: & \{0,1\} \to 0,\ 2 \to 1,\ \{5,7,9\} \to 2 \quad (x = 0) \\ \lfloor\min(2.32 + \max(3, h_1), 9.32)\rfloor: & h_1 \le 3 \to 5,\ 5 \to 7,\ \{7,9\} \to 9 \quad (x = 1), \end{cases}$$

i.e. shift the 2-bit code left and push in x.

Shift of h_0 (least structured). h_0's target values have no clean bit pattern, and the expression acts as a lookup table: the cubic P in h_1 is huge unless $h_1 \in \{0,1\}$ (and handles the single case where h_0' depends on h_0, via $\max(h_1, h_0)$); Q supplies $\{0,4,8\}$ for $h_1 \ge 2$, with 0.227 tuned so the rounded products have the right parity. This is the subproblem the other methods miss.

2 Alternating-last 4

Alternating-last 4

$h_0 \in \{0, 1, 6, 7, 8, 31, 32, 37, 38\}$, $h_1 \in \{0, 1\}$ (integer hidden-state coordinates; 11 joint states), $x \in \{0, 1\}$ (input bit), $y \in \{0, 1\}$ (output bit)

Scalar relation	Input-table size	Base	Evolved	MIPS-evolved	Discovered equation (MIPS-evolved)
$h_0' = f_0(h_0, h_1, x)$	22 state-input pairs	**1/10**	0/10	**5/10**	$\lfloor \mathrm{eq}(\mathrm{not}(h_0), x) + 30\,\mathrm{not}(x) - 0.203372\,h_0 + 7.82772 \rfloor$
$h_1' = f_1(h_0, h_1, x)$	22 state-input pairs	**1/10**	**10/10**	**10/10**	$\mathrm{zero}(\mathrm{xor}(\mathrm{lt}(x, h_0), x)) \cdot \max(h_0 - 37,\ \max(\mathrm{zero}(x), h_1))$
$y = g(h_0, h_1)$	11 states	**10/10**	**10/10**	**10/10**	h_1
Subproblems recovered across seeds		**3/3**	**2/3**	**3/3**	
All subproblems in the same seed		0/10	0/10	**5/10**	

Task. $y = 1$ iff the last 4 bits alternate (0101 or 1010).

What the state stores. The last bit and the length L of the current alternating run:

h_0	$6, 7, 8$	1	0	$31, 32$	$37\,(h_1{=}0)$	38	$37\,(h_1{=}1)$
last bit	1	1	1	0	0	0	0
L	1	2	≥ 3	1	2	3	≥ 4

Small h_0 $\Leftrightarrow$ last bit 1; large h_0 (31–38) $\Leftrightarrow$ last bit 0. $6, 7, 8$ and $31, 32$ are redundant copies with identical transitions. $h_1 = \mathbf{1}[L \geq 4]$, so $y = h_1$.

h_0': an affine "flip" map. Up to the small eq term, $h_0' \approx \lfloor 7.83 - h_0/5 + 30\,\mathbf{1}[x{=}0] \rfloor$.

- $x = 1$: large h_0 (run continues) $\to$ small: $31/32 \to 1$ ($L{=}2$), $37/38 \to 0$ ($L{\geq}3$); small h_0 (repeat, run resets) $\to 6/7/8$ ($L{=}1$).
- $x = 0$: $+30$ lands in the large band: large h_0 (repeat) $\to 31/32$; $6/7/8 \to 37$ ($L{=}2$), $1 \to 38$ ($L{=}3$). The $\mathrm{eq}(\mathrm{not}(h_0), x)$ term is the ± 1 correction for $h_0 = 0$ ($L{\geq}3 \to 37$ with $h_1' = 1$).

h_1': "run was already ≥ 3 and this bit continues it". $\mathrm{zero}(\mathrm{xor}(\mathrm{lt}(x, h_0), x)) = 1$ iff ($x{=}0$ and $h_0{=}0$) or ($x{=}1$ and $h_0 \geq 2$); then $\max(h_0 - 37, \max(\mathrm{zero}(x), h_1))$ keeps only the $L \geq 3$ states ($h_0 = 0$; or $h_0 = 38$ or $(37, h_1{=}1)$). h_1 is needed because $h_0 = 37$ alone does not separate $L = 2$ from $L \geq 4$.

3 Base-B addition: shared structure

Task. Each step reads one digit a, b of two base-B numbers (least significant first) and outputs the corresponding digit of the sum.

What the state stores. The full column sum $s = a + b + c_{\text{in}} \in \{0, \ldots, 2B-1\}$ ($2B$ states: 8, 10, 14), as two coordinates: a *carry bit* $\mathbf{1}[s \geq B]$ and a *digit coordinate* that decreases in s with a jump at the carry:

B	carry	digit coordinate
4	h_1	$h_0 = 5 - s + 2h_1$
5	h_1	$h_0 = 6 - s + 3h_1$
7	h_0	$h_1 = 8 - s + 5h_0$

Output $y = s \bmod B$: invert the digit coordinate and subtract B if the carry is set. **Carry update** $c' = \mathbf{1}[a + b + c \geq B]$: a threshold on $a + b$ plus carry. **Digit update**: compute $K - s' + \delta\,\mathbf{1}[s' \geq B]$ with $s' = a + b + c$, always as a max / min of a *linear branch* ($s' < B$) and a *mod branch* that implements the "subtract B" wrap-around.

4 Base-4 addition

Base-4 addition

$h_0 \in \{0, \ldots, 5\}$, $h_1 \in \{0, 1\}$ (integer hidden-state coordinates; 8 joint states), $a, b \in \{0, \ldots, 3\}$ (input digits), $y \in \{0, 1, 2, 3\}$ (sum digit)

Scalar relation	Input-table size	Base	Evolved	MIPS-evolved	Discovered equation (MIPS-evolved)
$h_0' = f_0(h_0, h_1, a, b)$	128 state-input pairs	0/10	**8/10**	**3/10**	$3.25534 - \min(\mathrm{mod}(-2.74466,\ a + b + h_1 - 1),\ a + b + h_1 - 1.74466)$
$h_1' = f_1(h_0, h_1, a, b)$	128 state-input pairs	**9/10**	**10/10**	**10/10**	$\mathrm{eq}(\min((a + b - 1.91107)(h_1 + 0.432432),\ 1.07176),\ 1)$
$y = g(h_0, h_1)$	8 states	**10/10**	**10/10**	**10/10**	$3 - h_0 + 2\,\mathrm{zero}(h_1)$
Subproblems recovered across seeds		**2/3**	**3/3**	**3/3**	
All subproblems in the same seed		0/10	**8/10**	**3/10**	

Carry $= h_1$, digit coordinate $h_0 = 5 - s + 2h_1$ ($s = 0..3 \mapsto 5, 4, 3, 2$; $s = 4..7 \mapsto 3, 2, 1, 0$).

Output. $y = 3 - h_0 + 2\,\mathrm{zero}(h_1)$: no carry gives $3 - (5 - s) + 2 = s$; carry gives $3 - (7 - s) = s - 4$.

Carry. $\mathrm{eq}(\min((a{+}b{-}1.91)(h_1{+}0.43), 1.07), 1)$ fires iff $(a{+}b{-}1.91)(h_1{+}0.43) > 0.5$: with $h_1 = 0$ this needs $a{+}b > 3.07$ (≥ 4); with $h_1 = 1$, $a{+}b > 2.26$ (≥ 3). That is $a + b + c \geq 4$, with the carry changing the slope rather than the offset.

Digit. With $s' = a + b + h_1$: $3.255 - \min(\mathrm{mod}(-2.745, s' - 1),\ s' - 1.745)$. Linear branch gives $5 - s'$ for $s' < 4$. For $s' \geq 4$, $\mathrm{mod}(-2.745, s' - 1) = s' - 3.745$, giving $7 - s'$ (the wrapped code). At $s' = 2, 3$ the two branches tie exactly.

5 Base-5 addition

Base-5 addition

$h_0 \in \{0, \ldots, 6\}$, $h_1 \in \{0, 1\}$ (integer hidden-state coordinates; 10 joint states), $a, b \in \{0, \ldots, 4\}$ (input digits), $y \in \{0, \ldots, 4\}$ (sum digit)

Scalar relation	Input-table size	Base	Evolved	MIPS-evolved	Discovered equation (MIPS-evolved)
$h_0' = f_0(h_0, h_1, a, b)$	250 state-input pairs	0/10	**2/10**	**3/10**	$\lfloor \max(6.06243 - (a+b+h_1),\ \mathrm{mod}(0.243552 - (a+b+h_1),\ 4.38026)) \rfloor$
$h_1' = f_1(h_0, h_1, a, b)$	250 state-input pairs	**8/10**	**10/10**	**10/10**	$\mathrm{eq}(1,\ \max(0.119279(a+b),\ \mathrm{mod}(-1.03325,\ 0.25413(a+b)h_1)))$
$y = g(h_0, h_1)$	10 states	**10/10**	**10/10**	**9/10**	$\lvert h_0 + h_1 - 6 \rvert - h_1$
Subproblems recovered across seeds		**2/3**	**3/3**	**3/3**	
All subproblems in the same seed		0/10	**2/10**	**3/10**	

Carry $= h_1$, digit coordinate $h_0 = 6 - s + 3h_1$ ($s = 0..4 \mapsto 6..2$; $s = 5..9 \mapsto 4..0$).

Output. $y = |h_0 + h_1 - 6| - h_1$: no carry gives $|-s| = s$; carry gives $|4 - s| - 1 = s - 5$.

Carry. $\mathrm{eq}(1, \max(0.119(a{+}b), \mathrm{mod}(-1.033, 0.254(a{+}b)h_1)))$. Without carry the mod term is 0 and $0.119(a{+}b) \in (0.5, 1.5)$ iff $a{+}b \geq 5$. With carry, the mod term lands at ≈ 1.0 for $a{+}b = 4$ and at 0.49 (just under the 0.5 cutoff) for $a{+}b \leq 3$. Correct, but a fragile modular coincidence.

Digit. With $s' = a + b + h_1$: $\lfloor \max(6.06 - s', \mathrm{mod}(0.24 - s', 4.38)) \rfloor$. For $s' \geq 5$, two periods (8.76) give mod $\approx 9 - s'$, the wrapped code; otherwise $6.06 - s'$ dominates.

6 Base-7 addition

Base-7 addition

$h_0 \in \{0, 1\}$, $h_1 \in \{0, \ldots, 8\}$ (integer hidden-state coordinates; 14 joint states), $a, b \in \{0, \ldots, 6\}$ (input digits), $y \in \{0, \ldots, 6\}$ (sum digit)

Scalar relation	Input-table size	Base	Evolved	MIPS-evolved	Discovered equation (MIPS-evolved)
$h_0' = f_0(h_0, h_1, a, b)$	686 state-input pairs	**10/10**	**10/10**	**10/10**	$\mathrm{eq}(0.0810486(a + b + h_0),\ 1)$
$h_1' = f_1(h_0, h_1, a, b)$	686 state-input pairs	0/10	**2/10**	**2/10**	$\lfloor \max(8.00009 - (a + b + h_0),\ \mathrm{mod}(13.0001,\ a + b + h_0)) \rfloor$
$y = g(h_0, h_1)$	14 states	**10/10**	**10/10**	**10/10**	$8 - h_1 - 2h_0$
Subproblems recovered across seeds		**2/3**	**3/3**	**3/3**	
All subproblems in the same seed		0/10	**2/10**	**2/10**	

Roles swap: carry $= h_0$, digit coordinate $h_1 = 8 - s + 5h_0$ ($s = 0..6 \mapsto 8..2$; $s = 7..13 \mapsto 6..0$).

Output. $y = 8 - h_1 - 2h_0$: s or $s - 7$.

Carry. $\mathrm{eq}(0.081(a + b + h_0), 1) = 1$ iff $a + b + h_0 \in (6.2, 18.5)$, i.e. exactly $a + b + c \geq 7$. The cleanest carry of the three.

Digit. $\lfloor \max(8 - s', \mathrm{mod}(13, s')) \rfloor$ with $s' = a + b + h_0$. For $s' \geq 7$, $13/s' < 2$ so $\mathrm{mod}(13, s') = 13 - s'$, the wrapped code; for $s' < 7$ the mod value is below $8 - s'$. So mod is used as a "subtract once if large" operator.

Takeaways

- The RNN representations are genuine algorithms: a shift register plus a running parity (parity), last bit plus run length (alternating), and column sum as (carry, reversed digit) (addition).
- Clean, interpretable recoveries: the parity rule $h_1 < 2h_0$, all outputs, and the base-4/base-7 carry thresholds.
- The remaining coordinates (parity h_0, alternating h_0/h_1, digit updates, base-5 carry) are correct piecewise lookups built from $\lfloor \cdot \rfloor$, mod and max, often exploiting numerical coincidences or the RNN's arbitrary, partly redundant lattice coordinates. These are exactly the subproblems baseline and SRBench-evolved PySR tend to miss.